OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
чате OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi- platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек- тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози- торий	Конкурен- ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Предметно ориентированные языки. Общие сведения

Цели вебинара

После занятия вы сможете

1. Знать элементы языка для построения DSL
2. Создавать собственные DSL
3. Применять DSL при разработке

Смысл

Зачем вам это уметь

1. DSL делает разработку более простой и наглядной
2. DSL используется везде в экосистеме Kotlin
3. Позволяет делать логику приложения более гибкой

Domain Specific Language

Domain Specific Language

DSL:

1. Аналог java-билдеров - Type-Safe Builders
2. Более удобный вариант построения объектов и конфигураций
3. Используется повсеместно в экосистеме Kotlin:
   a. Gradle Kotlin DSL
   b. KTOR
   c. kotlinx.html

https://kotlinlang.org/docs/reference/type-safe-builders.html

```
json {
    "name" to "Hitanshu"
    "age"  to 22
    "male" to true
    "address" to json {
        "line1" to "Flat No. 100"
        "line2" to "Triveni Apartments"
        "city"  to "New Delhi"
    }
}
```

Domain Specific Language

Преимущества:

- Поддержка проще чем у класс-утилит
- Читаемость лучше builder’ов
- Реализация проще изучения фреймворков

Элементы DSL

```
// Переопределение операторов
collection.add(element) // до
collection += element // после

// Соглашение для get/set методов
map.put("key", "value") // до
map["key"] = "value" // после

// Мульти-декларации
// до
val p = Point(0, 0)
val x1 = p.first
val y1 = p.second
// после
val (x, y) = Point(0, 0)
```

Элементы DSL

```
// Лямбда за скобками
list.forEach({ println(it) }) //  до
list.forEach { println(it) } // после

// Infix функции
1.to( that: "one") // до
val pair = 1 to "one" // после

// Extention функции
mylist.first()
// Лямбда с обработчиком
Person().apply { name = "John" }
```

Элементы DSL

Функция принимающая аргумент ***лямбда функция с объектом-приёмником*** (lambda function with receiver)

```
fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}
```

Обращение происходит через ключевое слово `this`.

```
html {
    this.head { /* ... */ }
    this.body { /* ... */ }
}
```

`this`, как обычно, можно опустить

```
html {
    head { /*...*/ }
    body { /*...*/ }
}
```

Kotlin Gradle DSL

```
android { this: BaseAppModuleExtension
    compileSdkVersion( apiLevel: 28)

    defaultConfig { this: DefaultConfig
        targetSdkVersion( targetSdkVersion: 28)
        minSdkVersion( minSdkVersion: 16)
        applicationId = "co.infinum.apprologic"
        versionName = "2.0.1"
        versionCode = 20
        multiDexEnabled = true

        javaCompileOptions { this: JavaCompileOptions
            annotationProcessorOptions { this: AnnotationProcessorOptions
                arguments = mapOf(
                    "classGeneratorPackageName"
                        to "co.infinum.apprologic.wrappers",
                    "wrapFactoryPackageName"
                        to "co.infinum.apprologic.wrappers.factory",
                    "jsHelperClass"
                        to "co.infinum.apprologic.helpers.JsHelper",
                    "wrapFactoryName"
                        to "CustomWrappersGen"
                )
            }
        }

        vectorDrawables.useSupportLibrary = true
    }
```

TeamCity DSL

app [~/work/app] - .../.teamcity/settings.kts [App_dsl]

Project

- app ~/work/app
  - .idea
  - .teamcity [App_dsl] sources root
    - App_dsl.iml
    - pom.xml
    - settings.kts
  - gradle
  - src
  - .gitignore
  - app.iml
  - build.gradle
  - gradlew
  - gradlew.bat
  - README.md
- External Libraries
- Scratches and Consoles
  - Extensions
  - JShell Console
  - Scratches

```
1  ...
8
9  version = "2018.1"
10
11 project { this: Project
12     buildType { this: BuildType
13         id( id: "Build")
14         name = "Build"
15
16         vcs { root(DslContext.settingsRoot) }
17
18         steps { this: BuildSteps
19             gradle { tasks = "clean build" }
20         }
21
22         artifactRules = "build/libs/app*.jar"
23
24         triggers { vcs {  } }
25
26         cleanup { this: Cleanup
27             artifacts(days = 2)
28             history(days = 3)
29         }
30     }
31 }
32
```

Особенности использования DSL

- Соответствует ООП, возможно переиспользование части DSL
- При неправильном проектировании, можно запутаться в this и it
- Может возникнуть достаточно глубокая вложенность
- Требуется документирование

```
fun main() {
    val person = buildPerson {
        age = 42
        greeting {
            "Hello" to "father"
            "Hi" to "mother"
            default = "What's up"
        }
```

LIVE

Слайд с заданием

1. Зачем нужен DSL?
2. На каких основных элементах Kotlin’a основывается DSL?

Слайд с заданием

В чем заключается разница функций a и b?

```
fun a(block: JsonObjectBuilder.() → Unit) {

}

fun b(block: (JsonObjectBuilder) → Unit) {

}
```

Слайд с заданием

Как можно запретить использование data в data?

```
fun c(block: JsonObjectBuilder.() → Unit) {
    schedule {  this: Cont
        data {
            data {}
        }
    }
}
```

Тезисы

Подведем итоги

1. DSL - это предметно ориентированный язык
2. Используется в качестве билдера для построения объектов со сложной логикой
3. Для создания используются lambda, infix, reified
4. DSL - хорошее решение в правильном месте
5. Предварительно проектируйте DSL

Домашнее задание модуля

для ознакомления, материалы к ДЗ будут выданы на последнем занятии модуля

1. создать три композитных проекта: (1) учебные модули; (2) плагин; (3) проектные модули
2. создать плагины Gradle для сборки подпроектов (1) на базе jvm; (2) на базе multiplatform
3. в плагины добавить такси для генерации докер-образов
4. создать файл libs.versions.toml и зацепить за него версии зависимостей в плагинах

Неофициальное ДЗ

1. Прочитать документацию по DSL на kotlinlang.org
2. Сделать простой DSL для генерации SQL запросов
   hw1-Sql.kt

Рефлексия

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

Модуль 2: Расширенные возможности Kotlin

Конкурентность с корутинами

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://kotlinlang.org
2. https://gradle.org
3. https://github.com/otuskotlin/202502-ok-marketplace
4. https://kotlinlang.org/docs/reference/type-safe-builders.html
5. https://habr.com/ru/company/haulmont/blog/341402
6. https://habr.com/ru/post/343730

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары